

Comments on why the test pass/fail are within the images. Also, didn't separate the code into 3 separate files, so it's easier to run tests

Bubble Sort

Faulty Code:

```
/**
 * Bubble Sort - Faulty Version
 * Outer loop runs  $i < n - 2$  instead of  $i < n - 1$ .
 * It has one less pass than required, so the two smallest
 * elements at the front of the array may never get fully sorted.
 */
public class BubbleSort { 4 usages

    public static void bubbleSort(int[] arr) { 4 usages
        int n = arr.length;
        for (int i = 0; i < n - 2; i++) { // FAULT: should be  $n - 1$ 
            for (int j = 0; j < n - i - 1; j++) {
                if (arr[j] > arr[j + 1]) {
                    int temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                }
            }
        }
    }
}
```

Test Cases:

```
@Test
public void testDoesNotExecuteFault1() {
    // Test 1: Already sorted array - passes because no swaps happen,
    // so the missing final pass has no effect
    int[] input = { 1, 2, 3, 4, 5, 6, 7 };
    int[] x = input.clone();
    int[] expected = { 1, 2, 3, 4, 5, 6, 7 };
    BubbleSort.bubbleSort(x);
    printResult(testName: "Test 1: Does Not Reveal Fault (Already Sorted)", input, expected, x);
    assertArrayEquals(expected, x);
}

@Test
public void testDoesNotExecuteFault2() {
    // Test 2: Partially sorted array - passes because elements happen to be
    // in correct positions after n-2 passes even though the final pass is missing
    int[] input = { 2, 1, 4, 3, 5 };
    int[] x = input.clone();
    int[] expected = { 1, 2, 3, 4, 5 };
    BubbleSort.bubbleSort(x);
    printResult(testName: "Test 2: Does Not Reveal Fault (Partially Sorted)", input, expected, x);
    assertArrayEquals(expected, x);
}

@Test
public void testExecutesFaultTest3() {
    // Test 3: Fully reversed array - fails because it needs all n-1 passes
    // to complete, and with only n-2 passes the first elements remain unsorted
    int[] input = { 7, 6, 5, 4, 3, 2, 1 };
    int[] x = input.clone();
    int[] expected = { 1, 2, 3, 4, 5, 6, 7 };
    BubbleSort.bubbleSort(x);
    printResult(testName: "Test 3: Shows Fault (Fully Reversed)", input, expected, x);
    assertArrayEquals(expected, x);
}

@Test
public void testExecutesFaultTest4() {
    // Test 4: Partially reversed array - fails because it also needs all n-1
    // passes to complete, leaving early elements unsorted
    int[] input = { 6, 7, 5, 4, 3, 2, 1 };
    int[] x = input.clone();
    int[] expected = { 1, 2, 3, 4, 5, 6, 7 };
    BubbleSort.bubbleSort(x);
    printResult(testName: "Test 4: Shows Fault (Worst Case)", input, expected, x);
    assertArrayEquals(expected, x);
}
```

Failing Tests:

```

  ▾ ✘ BubbleSortTest 20ms
    ✔ testDoesNotExecuteFault1 8ms
    ✔ testDoesNotExecuteFault2 0ms
    ✘ testExecutesFaultTest3 10ms
    ✘ testExecutesFaultTest4 2ms

  C:\Users\Andy\.jdk\openjdk-23.0.2\bin\java.exe ...
  =====
  Test 1: Does Not Reveal Fault (Already Sorted)
  =====
  Input:    [1, 2, 3, 4, 5, 6, 7]
  Expected: [1, 2, 3, 4, 5, 6, 7]
  Actual:   [1, 2, 3, 4, 5, 6, 7]
  Status:   PASS

  =====
  Test 2: Does Not Reveal Fault (Partially Sorted)
  =====
  Input:    [2, 1, 4, 3, 5]
  Expected: [1, 2, 3, 4, 5]
  Actual:   [1, 2, 3, 4, 5]
  Status:   PASS

  =====
  Test 3: Shows Fault (Fully Reversed)
  =====
  Input:    [7, 6, 5, 4, 3, 2, 1]
  Expected: [1, 2, 3, 4, 5, 6, 7]
  Actual:   [2, 1, 3, 4, 5, 6, 7]
  Status:   FAIL

```

```

  =====
  Test 4: Shows Fault (Worst Case)
  =====
  Input:    [6, 7, 5, 4, 3, 2, 1]
  Expected: [1, 2, 3, 4, 5, 6, 7]
  Actual:   [2, 1, 3, 4, 5, 6, 7]
  Status:   FAIL

```

Fixed Code:

```
/**
 *  Bubble Sort - Fixed Version
 *
 * * FIX: changed outer loop from i < n - 2 to i < n - 1 so that
 * * all n-1 passes are performed and every element reaches its correct position.
 */
public class BubbleSort { 4 usages

    public static void bubbleSort(int[] arr) { 4 usages
        int n = arr.length;
        for (int i = 0; i < n - 1; i++) { // FIXED: was n - 2
            for (int j = 0; j < n - i - 1; j++) {
                if (arr[j] > arr[j + 1]) {
                    int temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                }
            }
        }
    }
}
```

Passing Tests

```
✓ ✓ BubbleSortTest 12 ms
  ✓ testDoesNotExecuteFault 10 ms
  ✓ testDoesNotExecuteFault2 1 ms
  ✓ testExecutesFaultTest3 1 ms
  ✓ testExecutesFaultTest4 0 ms

✓ 4 tests passed 4 tests total, 12 ms

C:\Users\Andy\.jdk\openjdk-23.0.2\bin\java.exe ...
=====
Test 1: Does Not Reveal Fault (Already Sorted)
=====
Input:    [1, 2, 3, 4, 5, 6, 7]
Expected: [1, 2, 3, 4, 5, 6, 7]
Actual:   [1, 2, 3, 4, 5, 6, 7]
Status:   PASS

=====
Test 2: Does Not Reveal Fault (Partially Sorted)
=====
Input:    [2, 1, 4, 3, 5]
Expected: [1, 2, 3, 4, 5]
Actual:   [1, 2, 3, 4, 5]
Status:   PASS

=====
Test 3: Shows Fault (Fully Reversed)
=====
Input:    [7, 6, 5, 4, 3, 2, 1]
Expected: [1, 2, 3, 4, 5, 6, 7]
Actual:   [1, 2, 3, 4, 5, 6, 7]
Status:   PASS

=====
Test 4: Shows Fault (Worst Case)
=====
Input:    [6, 7, 5, 4, 3, 2, 1]
Expected: [1, 2, 3, 4, 5, 6, 7]
Actual:   [1, 2, 3, 4, 5, 6, 7]
Status:   PASS
```

Merge Sort

Faulty Code:

```
/**
 * Merge Sort - Faulty Version
 *
 * FAULT: the recursive call that sorts the right half is missing.
 * Only the left half is sorted; the right half remains in its original order.
 */
public class MergeSort { 4 usages

    public static void merge(int[] arr, int left, int mid, int right) { 2 usages
        int n1 = mid - left + 1;
        int n2 = right - mid;

        int[] L = new int[n1];
        int[] R = new int[n2];

        for (int i = 0; i < n1; i++)
            L[i] = arr[left + i];
        for (int j = 0; j < n2; j++)
            R[j] = arr[mid + 1 + j];

        int i = 0, j = 0, k = left;

        while (i < n1 && j < n2) {
            if (L[i] <= R[j]) {
                arr[k++] = L[i++];
            } else {
                arr[k++] = R[j++];
            }
        }

        while (i < n1) {
            arr[k++] = L[i++];
        }

        while (j < n2) {
            arr[k++] = R[j++];
        }
    }

    public static void mergeSort(int[] arr, int left, int right) { 7 usages
        if (left < right) {
            int mid = (left + right) / 2;
            mergeSort(arr, left, mid);
            // FAULT: missing mergeSort(arr, mid + 1, right)
            merge(arr, left, mid, right);
        }
    }
}
```

Test Cases:

```
public class MergeSortTest {
    @Test
    public void testDoesNotExecuteFault1() {
        // Test 1: Already sorted array - passes because the right half is already
        // sorted, so it never gets processed by the missing recursive call
        int[] input = { 1, 2, 3, 4, 5, 6, 7 };
        int[] x = input.clone();
        int[] expected = { 1, 2, 3, 4, 5, 6, 7 };
        MergeSort.mergeSort(x, left: 0, right: x.length - 1);
        printResult(testName: "Test 1: Does Not Reveal Fault (Already Sorted)", input, expected, x);
        assertArrayEquals(expected, x);
    }

    @Test
    public void testDoesNotExecuteFault2() {
        // Test 2: Partially reversed array - passes because when the right half
        // [3] is skipped, merging [1,2] with [3] still produces the correct result
        int[] input = { 2, 1, 3 };
        int[] x = input.clone();
        int[] expected = { 1, 2, 3 };
        MergeSort.mergeSort(x, left: 0, right: x.length - 1);
        printResult(testName: "Test 2: Does Not Reveal Fault (Partially Reversed)", input, expected, x);
        assertArrayEquals(expected, x);
    }

    @Test
    public void testExecutesFaultTest3() {
        // Test 3: Fully reversed array - fails because the right half is unsorted
        // and the missing recursive call never sorts it, resulting in an incorrectly
        // merged final array
        int[] input = { 7, 6, 5, 4, 3, 2, 1 };
        int[] x = input.clone();
        int[] expected = { 1, 2, 3, 4, 5, 6, 7 };
        MergeSort.mergeSort(x, left: 0, right: x.length - 1);
        printResult(testName: "Test 3: Shows Fault (Fully Reversed)", input, expected, x);
        assertArrayEquals(expected, x);
    }

    @Test
    public void testExecutesFaultTest4() {
        // Test 4: Random Numbers with unsorted right half - fails because the missing
        // recursive call doesn't sort the right half, leaving it unsorted for the final
        // merge
        int[] input = { 5, 3, 8, 1, 9, 2, 7 };
        int[] x = input.clone();
        int[] expected = { 1, 2, 3, 5, 7, 8, 9 };
        MergeSort.mergeSort(x, left: 0, right: x.length - 1);
        printResult(testName: "Test 4: Shows Fault (Random Numbers)", input, expected, x);
        assertArrayEquals(expected, x);
    }
}
```

Failing Tests:

```

✓ MergeSortTest 20 ms
  ✓ testDoesNotExecuteFault1 8 ms
  ✓ testDoesNotExecuteFault2 0 ms
  ✗ testExecutesFaultTest3 10 ms
  ✗ testExecutesFaultTest4 2 ms

✗ 2 tests failed, 2 passed 4 tests total, 20 ms
C:\Users\Andy\.jdk\openjdk-23.0.2\bin\java.exe ...
=====
Test 1: Does Not Reveal Fault (Already Sorted)
=====
Input:    [1, 2, 3, 4, 5, 6, 7]
Expected: [1, 2, 3, 4, 5, 6, 7]
Actual:   [1, 2, 3, 4, 5, 6, 7]
Status:   PASS

=====
Test 2: Does Not Reveal Fault (Partially Reversed)
=====
Input:    [2, 1, 3]
Expected: [1, 2, 3]
Actual:   [1, 2, 3]
Status:   PASS

=====
Test 3: Shows Fault (Fully Reversed)
=====
Input:    [7, 6, 5, 4, 3, 2, 1]
Expected: [1, 2, 3, 4, 5, 6, 7]
Actual:   [3, 2, 1, 5, 4, 6, 7]
Status:   FAIL

```

```

=====
Test 4: Shows Fault (Random Numbers)
=====
Input:    [5, 3, 8, 1, 9, 2, 7]
Expected: [1, 2, 3, 5, 7, 8, 9]
Actual:   [3, 5, 8, 1, 9, 2, 7]
Status:   FAIL

```


Fixed Code:

```
/**
 * Merge Sort - Fixed Version
 *
 * FIX: added the missing recursive call mergeSort(arr, mid + 1, right)
 * so that both the left and right halves are fully sorted before merging.
 */
public class MergeSort { 4 usages

    public static void merge(int[] arr, int left, int mid, int right) { 2 usages
        int n1 = mid - left + 1;
        int n2 = right - mid;

        int[] L = new int[n1];
        int[] R = new int[n2];

        for (int i = 0; i < n1; i++)
            L[i] = arr[left + i];
        for (int j = 0; j < n2; j++)
            R[j] = arr[mid + 1 + j];

        int i = 0, j = 0, k = left;

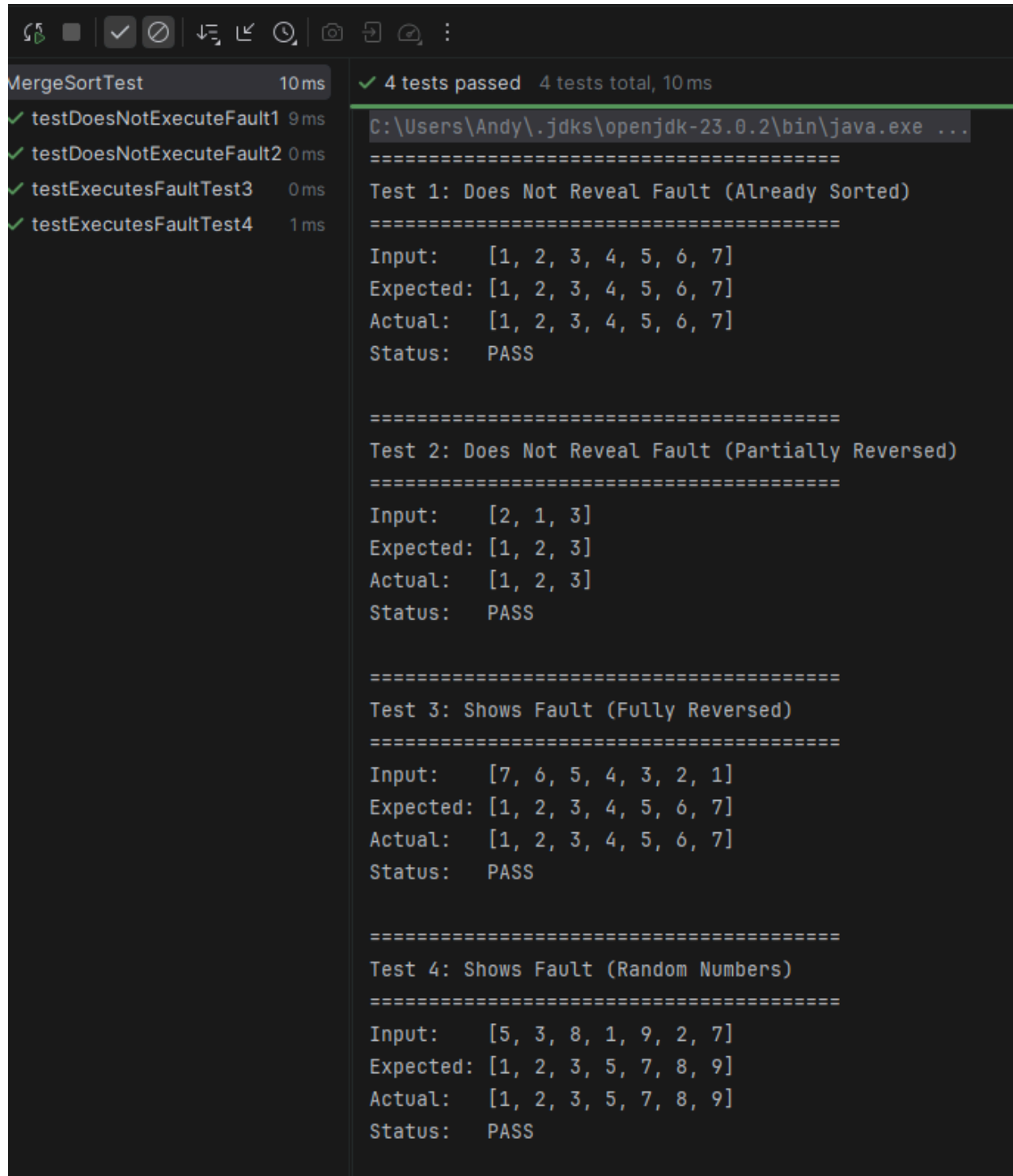
        while (i < n1 && j < n2) {
            if (L[i] <= R[j]) {
                arr[k++] = L[i++];
            } else {
                arr[k++] = R[j++];
            }
        }

        while (i < n1) {
            arr[k++] = L[i++];
        } // FIXED: was missing

        while (j < n2) {
            arr[k++] = R[j++];
        }
    }

    public static void mergeSort(int[] arr, int left, int right) { 8 usages
        if (left < right) {
            int mid = (left + right) / 2;
            mergeSort(arr, left, mid);
            mergeSort(arr, left: mid + 1, right);
            merge(arr, left, mid, right);
        }
    }
}
```

Passing Tests



```

MergeSortTest 10ms
✓ testDoesNotExecuteFault1 9ms
✓ testDoesNotExecuteFault2 0ms
✓ testExecutesFaultTest3 0ms
✓ testExecutesFaultTest4 1ms

✓ 4 tests passed 4 tests total, 10ms

C:\Users\Andy\.jdk\openjdk-23.0.2\bin\java.exe ...
=====
Test 1: Does Not Reveal Fault (Already Sorted)
=====
Input:      [1, 2, 3, 4, 5, 6, 7]
Expected:   [1, 2, 3, 4, 5, 6, 7]
Actual:     [1, 2, 3, 4, 5, 6, 7]
Status:     PASS

=====
Test 2: Does Not Reveal Fault (Partially Reversed)
=====
Input:      [2, 1, 3]
Expected:   [1, 2, 3]
Actual:     [1, 2, 3]
Status:     PASS

=====
Test 3: Shows Fault (Fully Reversed)
=====
Input:      [7, 6, 5, 4, 3, 2, 1]
Expected:   [1, 2, 3, 4, 5, 6, 7]
Actual:     [1, 2, 3, 4, 5, 6, 7]
Status:     PASS

=====
Test 4: Shows Fault (Random Numbers)
=====
Input:      [5, 3, 8, 1, 9, 2, 7]
Expected:   [1, 2, 3, 5, 7, 8, 9]
Actual:     [1, 2, 3, 5, 7, 8, 9]
Status:     PASS

```

Quick Sort

Faulty Code:

```
QuickSortTest.java x  Faulty code\QuickSort.java x  Faulty code\BubbleSort.java  Fixed c
  /**
   * Quick Sort - Faulty Version
   * 
   * FAULT: the recursive call for the right sub-array is missing.
   * This means the left side of the array will be sorted but
   * Not right one.
   */
  public class QuickSort { 4 usages

    public static int partition(int[] arr, int low, int high) { 2 usages
      int pivot = arr[high];
      int i = low - 1;
      for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
          i++;
          int temp = arr[i];
          arr[i] = arr[j];
          arr[j] = temp;
        }
      }
      int temp = arr[i + 1];
      arr[i + 1] = arr[high];
      arr[high] = temp;
      return i + 1;
    }

    public static void quickSort(int[] arr, int low, int high) { 7 usages
      if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, high: pi - 1);
        // FAULT: missing quickSort(arr, pi + 1, high)
      }
    }
  }
}
```

Test Cases:

```
public class QuickSortTest {

    public void testDoesNotExecuteFault1() {
        // Test 1: Sorted ascending array - passes because the pivot is always the
        // maximum, so the right partition is always empty and the missing recursive
        // call has no work
        int[] input = { 1, 2, 3, 4, 5, 6, 7 };
        int[] x = input.clone();
        int[] expected = { 1, 2, 3, 4, 5, 6, 7 };
        QuickSort.quickSort(x, low: 0, high: x.length - 1);
        printResult(testName: "Test 1: Does Not Reveal Fault (Already Sorted)", input, expected, x);
        assertArrayEquals(expected, x);
    }

    @Test
    public void testDoesNotExecuteFault2() {
        // Test 2: Partially sorted array - passes because the pivot ends up being the
        // maximum, making the right subarray empty so the missing recursive call does
        // nothing
        int[] input = { 2, 1, 3 };
        int[] x = input.clone();
        int[] expected = { 1, 2, 3 };
        QuickSort.quickSort(x, low: 0, high: x.length - 1);
        printResult(testName: "Test 2: Does Not Reveal Fault (Partially Sorted)", input, expected, x);
        assertArrayEquals(expected, x);
    }

    @Test
    public void testExecutesFaultTest3() {
        // Test 3: Fully reversed array - fails because the pivot is always the minimum,
        // so everything goes to the right partition which never gets sorted due to
        // the missing recursive call
        int[] input = { 7, 6, 5, 4, 3, 2, 1 };
        int[] x = input.clone();
        int[] expected = { 1, 2, 3, 4, 5, 6, 7 };
        QuickSort.quickSort(x, low: 0, high: x.length - 1);
        printResult(testName: "Test 3: Shows Fault (Fully Reversed)", input, expected, x);
        assertArrayEquals(expected, x);
    }

    @Test
    public void testExecutesFaultTest4() {
        // Test 4: Random numbers - fails because it creates non-empty right partitions
        // with unsorted elements that never get sorted without the missing recursive
        // call
        int[] input = { 3, 1, 4, 2, 6, 5, 7 };
        int[] x = input.clone();
        int[] expected = { 1, 2, 3, 4, 5, 6, 7 };
        QuickSort.quickSort(x, low: 0, high: x.length - 1);
        printResult(testName: "Test 4: Shows Fault (Random numbers)", input, expected, x);
        assertArrayEquals(expected, x);
    }
}
```

Failing Tests:

```
C:\Users\Andy\.jdk\openjdk-23.0.2\bin\java.exe ...  
=====
```

Test 1: Does Not Reveal Fault (Already Sorted)

```
=====
```

Input: [1, 2, 3, 4, 5, 6, 7]
Expected: [1, 2, 3, 4, 5, 6, 7]
Actual: [1, 2, 3, 4, 5, 6, 7]
Status: PASS

```
=====
```

Test 2: Does Not Reveal Fault (Partially Sorted)

```
=====
```

Input: [2, 1, 3]
Expected: [1, 2, 3]
Actual: [1, 2, 3]
Status: PASS

```
=====
```

Test 3: Shows Fault (Fully Reversed)

```
=====
```

Input: [7, 6, 5, 4, 3, 2, 1]
Expected: [1, 2, 3, 4, 5, 6, 7]
Actual: [1, 6, 5, 4, 3, 2, 7]
Status: FAIL

```
=====
```

Test 4: Shows Fault (Random numbers)

```
=====
```

Input: [3, 1, 4, 2, 6, 5, 7]
Expected: [1, 2, 3, 4, 5, 6, 7]
Actual: [1, 2, 4, 3, 5, 6, 7]
Status: FAIL

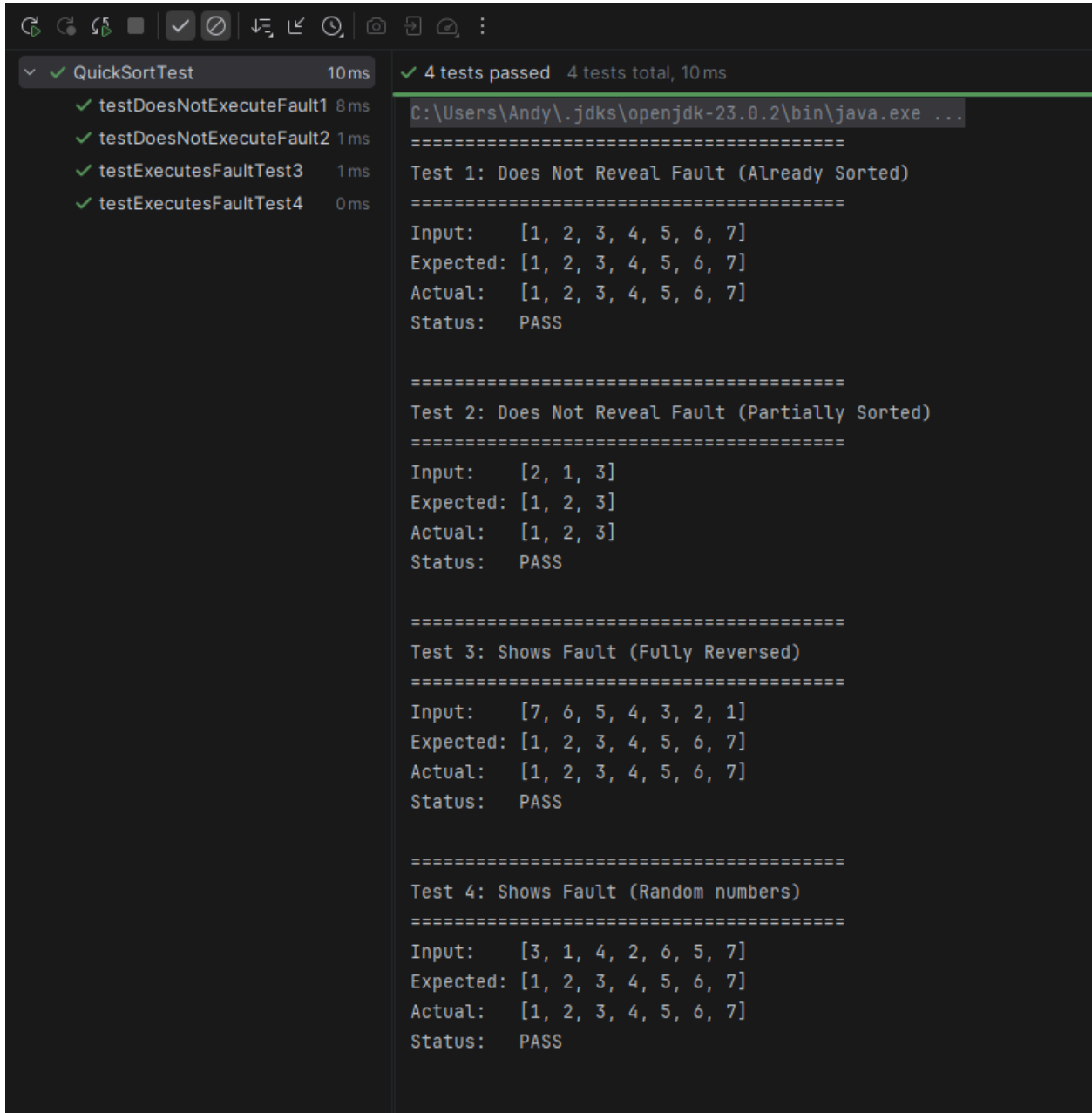
Fixed Code:

```
/**
 * Quick Sort - Fixed Version
 *
 * FIX: added the missing recursive call quickSort(arr, pi + 1, high)
 * so that both the left and right sub-arrays are fully sorted.
 */
public class QuickSort { 4 usages

    public static int partition(int[] arr, int low, int high) { 2 usages
        int pivot = arr[high];
        int i = low - 1;
        for (int j = low; j < high; j++) {
            if (arr[j] <= pivot) {
                i++;
                int temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }
        int temp = arr[i + 1];
        arr[i + 1] = arr[high];
        arr[high] = temp;
        return i + 1;
    }

    public static void quickSort(int[] arr, int low, int high) { 7 usages
        if (low < high) {
            int pi = partition(arr, low, high);
            quickSort(arr, low, high: pi - 1);
            quickSort(arr, low: pi + 1, high); // FIXED: was missing
        }
    }
}
```

Passing Tests



```
C:\Users\Andy\.jdk\openjdk-23.0.2\bin\java.exe ...  
=====  
Test 1: Does Not Reveal Fault (Already Sorted)  
=====  
Input:    [1, 2, 3, 4, 5, 6, 7]  
Expected: [1, 2, 3, 4, 5, 6, 7]  
Actual:   [1, 2, 3, 4, 5, 6, 7]  
Status:   PASS  
  
=====  
Test 2: Does Not Reveal Fault (Partially Sorted)  
=====  
Input:    [2, 1, 3]  
Expected: [1, 2, 3]  
Actual:   [1, 2, 3]  
Status:   PASS  
  
=====  
Test 3: Shows Fault (Fully Reversed)  
=====  
Input:    [7, 6, 5, 4, 3, 2, 1]  
Expected: [1, 2, 3, 4, 5, 6, 7]  
Actual:   [1, 2, 3, 4, 5, 6, 7]  
Status:   PASS  
  
=====  
Test 4: Shows Fault (Random numbers)  
=====  
Input:    [3, 1, 4, 2, 6, 5, 7]  
Expected: [1, 2, 3, 4, 5, 6, 7]  
Actual:   [1, 2, 3, 4, 5, 6, 7]  
Status:   PASS
```